

```

201 def main():
202     print("=" * 70)
203     print("Q2: MATRIX DECOMPOSITIONS")
204     print("=" * 70)
205
206     # Q2(a): LU Decomposition
207     print("\n" + "=" * 70)
208     print("Q2(a): LU Decomposition via Elementary Matrices")
209     print("=" * 70)
210
211     # Fixed seed for reproducible output - matches handwritten submission
212     np.random.seed(123)
213     matrix_dimension = 4
214     spd_matrix = generate_spd_matrix(matrix_dimension)
215
216     print_matrix(spd_matrix, "Symmetric Positive Definite Matrix A", decimals=3)
217
218     # Perform LU decomposition with verbose output showing all elementary matrices
219     lower_matrix, upper_matrix, elementary_matrices = lu_decomposition_elementary(spd_matrix, verbose=True)
220
221     print("\n" + "=" * 70)
222     print("SUMMARY - Final LU Decomposition Matrices:")
223     print("=" * 70)
224     print_matrix(lower_matrix, "Lower triangular matrix L", decimals=3)
225     print_matrix(upper_matrix, "Upper triangular matrix U", decimals=3)
226     print(f"\nNumber of elementary matrices used: {len(elementary_matrices)}")
227
228     # Verify A = LU
229     lu_product = matrix_multiply(lower_matrix, upper_matrix)
230     lu_verification_error = 0
231     for row_idx in range(matrix_dimension):
232         for col_idx in range(matrix_dimension):
233             reconstruction_diff = abs(spd_matrix[row_idx][col_idx] - lu_product[row_idx][col_idx])
234             lu_verification_error = max(lu_verification_error, reconstruction_diff)
235
236     print(f"\nVerification A = LU:")
237     print(f"    Maximum difference: {format_number(lu_verification_error)}")
238     print(f"    Decomposition valid: {'YES' if lu_verification_error < 1e-9 else 'NO'}")
239
240     print("\nLogical Explanation (Q2a):")
241     print("    • LU decomposes  $A = LU$  where L=lower triangular, U=upper triangular")
242     print("    • We use elementary matrices  $E_k$  to eliminate below-diagonal elements")
243     print("    • Each  $E_k$  performs:  $\text{row}_i = \text{row}_i - \text{factor} \times \text{row}_j$ ")
244     print("    • After all eliminations:  $E_n \dots E_2 \cdot E_1 \cdot A = U$  (upper triangular)")
245     print("    • Therefore:  $A = E_1^{-1} \cdot E_2^{-1} \dots E_n^{-1} \cdot U = L \cdot U$ ")
246     print("    • L is built from inverses of elementary matrices (just flip signs)")
247
248

```

```

# Q2(c): QR Decomposition (example)
print("\n" + "=" * 70)
print("Q2(c): QR Decomposition (Example with 5x3)")
print("=" * 70)

np.random.seed(456)
qr_rows, qr_cols = 5, 3
qr_test_matrix = np.random.uniform(-5, 5, (qr_rows, qr_cols)).tolist()

print_matrix(qr_test_matrix, f"Matrix A ({qr_rows}x{qr_cols})", decimals=3)

orthogonal_matrix, upper_triangular_r = qr_decomposition(qr_test_matrix)

print_matrix(orthogonal_matrix, "Orthonormal matrix Q", decimals=4)
print_matrix(upper_triangular_r, "Upper triangular matrix R", decimals=4)

# Verify A = QR
qr_product = matrix_multiply(orthogonal_matrix, upper_triangular_r)
qr_verification_error = 0
for row_idx in range(qr_rows):
    for col_idx in range(qr_cols):
        qr_diff = abs(qr_test_matrix[row_idx][col_idx] - qr_product[row_idx][col_idx])
        qr_verification_error = max(qr_verification_error, qr_diff)

print(f"\nVerification A = QR:")
print(f"    Maximum difference: {format_number(qr_verification_error)}")

# Verify Q^T * Q = I
q_transpose = [[orthogonal_matrix[col_idx][row_idx] for col_idx in range(qr_rows)] for row_idx in range(qr_cols)]
q_transpose_q = matrix_multiply(q_transpose, orthogonal_matrix)
print_matrix(q_transpose_q, "Q^T·Q (should be identity)", decimals=4)

print("\nLogical Explanation (Q2c):")
print("    • QR decomposes  $A = Q \cdot R$  where  $Q$ =orthogonal,  $R$ =upper triangular")
print("    • Uses Gram-Schmidt: each column becomes orthogonal to previous ones")
print("    • Step 1:  $u_j = a_j - \text{proj}(a_j \text{ onto } q_1) - \dots - \text{proj}(a_j \text{ onto } q_{j-1})$ ")
print("    • Step 2:  $q_j = u_j / ||u_j||$  (normalize to unit length)")
print("    •  $R[i][j]$  stores the coefficients:  $R[i][j] = q_i \cdot a_j$ ")
print("    •  $Q$  has orthonormal columns:  $Q^T \cdot Q = I$  (verified above)")

```

```

# Q2(d): QR with 7x5 matrix
print("\n" + "=" * 70)
print("Q2(d): QR Decomposition with Random 7x5 Matrix")
print("=" * 70)

np.random.seed(789)
large_qr_rows, large_qr_cols = 7, 5
large_qr_matrix = np.random.uniform(-8, 8, (large_qr_rows, large_qr_cols)).tolist()

print_matrix(large_qr_matrix, "Random 7x5 Matrix A", decimals=2)

large_q_matrix, large_r_matrix = qr_decomposition(large_qr_matrix)

print_matrix(large_q_matrix, "Q (7x5)", decimals=4)
print_matrix(large_r_matrix, "R (5x5 upper triangular)", decimals=4)

print("\nDiagonal elements of R:")
r_diagonal_elements = [large_r_matrix[diag_idx][diag_idx] for diag_idx in range(large_qr_cols)]
print([round(diagonal_val, 4) for diagonal_val in r_diagonal_elements])

print("\nObservation on Diagonal Elements of R:")
print(" ALL diagonal elements of R are STRICTLY POSITIVE.")
print(" This is a fundamental property of QR decomposition when columns are linearly independent.")
print("")
print(" Mathematical Explanation:")
print(" • Each R[j][j] represents ||u_j|| where u_j is the residual vector")
print(" • u_j = a_j - proj_1 - proj_2 - ... - proj_{j-1}")
print(" • proj_k means the projection of column a_j onto orthonormal vector q_k")
print(" • Since columns are linearly independent, u_j ≠ 0, so ||u_j|| > 0")
print("")
print(" Physical Interpretation:")
print(" • Gram-Schmidt finds what's 'new' in each column after removing")
print("   everything that can be expressed using previous columns")
print(" • If R[j][j] = 0, it would mean column j adds nothing new")
print(" • This would violate linear independence")
print("")
print(" Practical Consequences:")
print(" • R is invertible (no zero diagonal elements)")
print(" • QR decomposition succeeded without numerical breakdown")
print(" • Original matrix A has full column rank")

# Verify A = QR
large_qr_product = matrix_multiply(large_q_matrix, large_r_matrix)
large_qr_verification_error = 0
for row_idx in range(large_qr_rows):
    for col_idx in range(large_qr_cols):
        large_qr_diff = abs(large_qr_matrix[row_idx][col_idx] - large_qr_product[row_idx][col_idx])
        large_qr_verification_error = max(large_qr_verification_error, large_qr_diff)

```

```

# Q2(b): Cholesky Decomposition
print("\n" + "=" * 70)
print("Q2(b): Cholesky Decomposition")
print("=" * 70)

print("Using the same SPD matrix from Q2(a)")

cholesky_factor = cholesky_decomposition(spd_matrix)
print_matrix(cholesky_factor, "Cholesky factor L", decimals=4)

# Compute L^T
cholesky_transpose = [[cholesky_factor[col_idx][row_idx] for col_idx in range(matrix_dimension)] for row_idx in range(matrix_dimension)]

# Verify A = L * L^T
cholesky_product = matrix_multiply(cholesky_factor, cholesky_transpose)
cholesky_verification_error = 0
for row_idx in range(matrix_dimension):
    for col_idx in range(matrix_dimension):
        cholesky_diff = abs(spd_matrix[row_idx][col_idx] - cholesky_product[row_idx][col_idx])
        cholesky_verification_error = max(cholesky_verification_error, cholesky_diff)

print(f"\nVerification A = L·L^T:")
print(f"  Maximum difference: {format_number(cholesky_verification_error)}")
print(f"  Decomposition valid: {'YES' if cholesky_verification_error < 1e-9 else 'NO'}")

print("\nLogical Explanation (Q2b):")
print("  • Cholesky works only for Symmetric Positive Definite (SPD) matrices")
print("  • It finds L such that A = L·L^T (like taking 'square root' of matrix)")
print("  • More efficient than LU: only need to compute lower triangle")
print("  • Formula:  $L[i][j] = (A[i][j] - \sum L[i][k] \cdot L[j][k]) / L[j][j]$ ")
print("  • SPD guarantee ensures all diagonal elements remain positive")

```

```

126 def cholesky_decomposition(input_matrix):
148
149     return cholesky_factor
150
151
152 def qr_decomposition(input_matrix):
153     """
154     QR decomposition using Gram-Schmidt orthogonalization
155     A = Q * R where Q has orthonormal columns, R is upper triangular
156     """
157     num_rows = len(input_matrix)
158     num_cols = len(input_matrix[0])
159
160     orthonormal_matrix = [[0.0] * num_cols for _ in range(num_rows)]
161     upper_triangular_matrix = [[0.0] * num_cols for _ in range(num_cols)]
162
163     for current_col in range(num_cols):
164         # Start with column j of A
165         working_vector = [input_matrix[row_idx][current_col] for row_idx in range(num_rows)]
166
167         # Subtract projections onto all previous orthonormal vectors
168         for prev_col in range(current_col):
169             # Compute projection coefficient (dot product of A[:,j] with Q[:,i])
170             projection_coefficient = sum(input_matrix[row_idx][current_col] * orthonormal_matrix[row_idx][prev_col]
171                                         for row_idx in range(num_rows))
172             upper_triangular_matrix[prev_col][current_col] = projection_coefficient
173
174             # Subtract projection: v = v - projection_coeff * q_i
175             for row_idx in range(num_rows):
176                 working_vector[row_idx] -= projection_coefficient * orthonormal_matrix[row_idx][prev_col]
177
178         # Compute norm of the orthogonal residual
179         vector_norm = sum(working_vector[row_idx] ** 2 for row_idx in range(num_rows)) ** 0.5
180         upper_triangular_matrix[current_col][current_col] = vector_norm
181
182         # Normalize to get orthonormal vector q_j
183         if vector_norm > 1e-9:
184             for row_idx in range(num_rows):
185                 orthonormal_matrix[row_idx][current_col] = working_vector[row_idx] / vector_norm
186
187     return orthonormal_matrix, upper_triangular_matrix
188
189
190 def print_matrix(matrix, label="Matrix", decimals=3):
191     """Pretty print a matrix"""
192     print(f"\n{label}:")
193     for row in matrix:
194         print([round(val, decimals) for val in row])
195
196
197 # =====

```

```

100 def lu_decomposition_elementary(input_matrix, verbose=False):
107     if verbose:
108         print("\n" + "=" * 70)
109         print("FINAL RESULT:")
110         print("\nLower triangular matrix L (with 1s on diagonal):")
111         print_matrix(lower_triangular, "", decimals=3)
112         print("\nUpper triangular matrix U:")
113         print_matrix(upper_triangular, "", decimals=3)
114         print("\nNote: L is constructed from inverses of elementary matrices")
115         print("      L[i][j] = factor means we used E that subtracted factor*R_j from R_i")
116         print("=" * 70)
117
118     # Build L from inverses of elementary matrices
119     # Inverse of E just flips the sign: -factor becomes +factor
120     for elementary_matrix, target_row, pivot_row, factor in elementary_matrices_list:
121         lower_triangular[target_row][pivot_row] = factor
122
123     return lower_triangular, upper_triangular, elementary_matrices_list
124
125
126 def cholesky_decomposition(input_matrix):
127     """
128     Cholesky decomposition:  $A = L * L^T$ 
129     Returns L (lower triangular)
130     """
131     matrix_size = len(input_matrix)
132     cholesky_factor = [[0.0] * matrix_size for _ in range(matrix_size)]
133
134     for current_row in range(matrix_size):
135         for current_col in range(current_row + 1):
136             if current_row == current_col:
137                 # Diagonal element
138                 sum_of_squares = sum(cholesky_factor[current_row][prev_col] ** 2 for prev_col in range(current_col))
139                 diagonal_value = input_matrix[current_row][current_row] - sum_of_squares
140                 if diagonal_value < 0:
141                     raise ValueError(f"Matrix not positive definite at ({current_row},{current_row})")
142                 cholesky_factor[current_row][current_col] = diagonal_value ** 0.5
143             else:
144                 # Below diagonal element
145                 sum_of_products = sum(cholesky_factor[current_row][prev_col] * cholesky_factor[current_col][prev_col]
146                                     for prev_col in range(current_col))
147                 cholesky_factor[current_row][current_col] = (input_matrix[current_row][current_col] - sum_of_products) / cholesky_factor[current_row][current_col]
148
149     return cholesky_factor
150
151
152 def qr_decomposition(input_matrix):
153     """
154     QR decomposition using Gram-Schmidt orthogonalization
155     A = Q * R where Q has orthonormal columns, R is upper triangular
156     """

```



```

60 def lu_decomposition_elementary(input_matrix, verbose=False):
61     """
62     LU decomposition using elementary matrices
63     Returns L and U such that A = LU
64     Also returns list of elementary matrices for verification
65     """
66     matrix_size = len(input_matrix)
67     upper_triangular = [row[:] for row in input_matrix] # Copy of A
68     lower_triangular = [[0.0] * matrix_size for _ in range(matrix_size)]
69     for diagonal_idx in range(matrix_size):
70         lower_triangular[diagonal_idx][diagonal_idx] = 1.0 # Start with identity
71
72     elementary_matrices_list = []
73
74     if verbose:
75         print("\nSTEP-BY-STEP LU DECOMPOSITION USING ELEMENTARY MATRICES")
76         print("=" * 70)
77         print("\nStarting matrix A:")
78         print_matrix(upper_triangular, "", decimals=3)
79         step_count = 1
80
81     # Forward elimination
82     for pivot_col in range(matrix_size): # For each column
83         for elimination_row in range(pivot_col + 1, matrix_size): # For each row below diagonal
84             if abs(upper_triangular[pivot_col][pivot_col]) < 1e-9:
85                 continue # Skip if pivot is zero (shouldn't happen for SPD)
86
87             elimination_factor = upper_triangular[elimination_row][pivot_col] / upper_triangular[pivot_col][pivot_col]
88
89             # Create and store elementary matrix
90             elementary_matrix = create_elementary_matrix(matrix_size, elimination_row, pivot_col, elimination_factor)
91             elementary_matrices_list.append((elementary_matrix, elimination_row, pivot_col, elimination_factor))
92
93             if verbose:
94                 print(f"\n--- Step {step_count} ---")
95                 print(f"Operation: R{elimination_row+1} = R{elimination_row+1} - ({elimination_factor:.3f}) x R{pivot_col+1}")
96                 print(f"\nElementary matrix E{step_count}:")
97                 print_matrix(elementary_matrix, "", decimals=3)
98                 step_count += 1
99
100             # Apply elementary matrix to U
101             upper_triangular = matrix_multiply(elementary_matrix, upper_triangular)
102
103             if verbose:
104                 print(f"\nResulting matrix after E{step_count-1} x A:")
105                 print_matrix(upper_triangular, "", decimals=3)
106
107     if verbose:
108         print("\n" + "=" * 70)
109         print("FINAL RESULT:")

```

```

9 import numpy as np
10
11
12 def format_number(num, decimals=2):
13     """Format number to avoid scientific notation for small values"""
14     if abs(num) < 1e-9:
15         return "0.0"
16     elif abs(num) < 1e-3:
17         return f"{num:.{decimals+2}f}".rstrip('0').rstrip('.')
18     else:
19         return f"{num:.{decimals}e}"
20
21
22 def generate_spd_matrix(matrix_size):
23     """
24     Generate a symmetric positive definite matrix
25     Using A = B^T * B + n*I method
26     """
27     random_matrix = np.random.uniform(-3, 3, (matrix_size, matrix_size))
28
29     # Compute B^T * B + n*I for guaranteed positive definiteness
30     spd_matrix = random_matrix.T @ random_matrix + matrix_size * np.eye(matrix_size)
31
32     return spd_matrix.tolist()
33
34
35 def create_elementary_matrix(matrix_size, target_row, pivot_row, elimination_factor):
36     """
37     Create elementary matrix that performs: row_i = row_i - factor * row_j
38     This is identity matrix with E[i][j] = -factor
39     """
40     elementary_matrix = [[0.0] * matrix_size for _ in range(matrix_size)]
41     for diagonal_index in range(matrix_size):
42         elementary_matrix[diagonal_index][diagonal_index] = 1.0
43     elementary_matrix[target_row][pivot_row] = -elimination_factor
44     return elementary_matrix
45
46
47 def matrix_multiply(matrix_a, matrix_b):
48     """Multiply two matrices A and B"""
49     rows_a = len(matrix_a)
50     cols_b = len(matrix_b[0])
51     cols_a_rows_b = len(matrix_b)
52     product_matrix = [[0.0] * cols_b for _ in range(rows_a)]
53     for row_idx in range(rows_a):
54         for col_idx in range(cols_b):
55             for inner_idx in range(cols_a_rows_b):
56                 product_matrix[row_idx][col_idx] += matrix_a[row_idx][inner_idx] * matrix_b[inner_idx][col_idx]
57     return product_matrix
58
59

```